

Test d'IHM et langage Java

Test du jeudi 18 juin 2026 - Durée 2 heures - Documents non autorisés

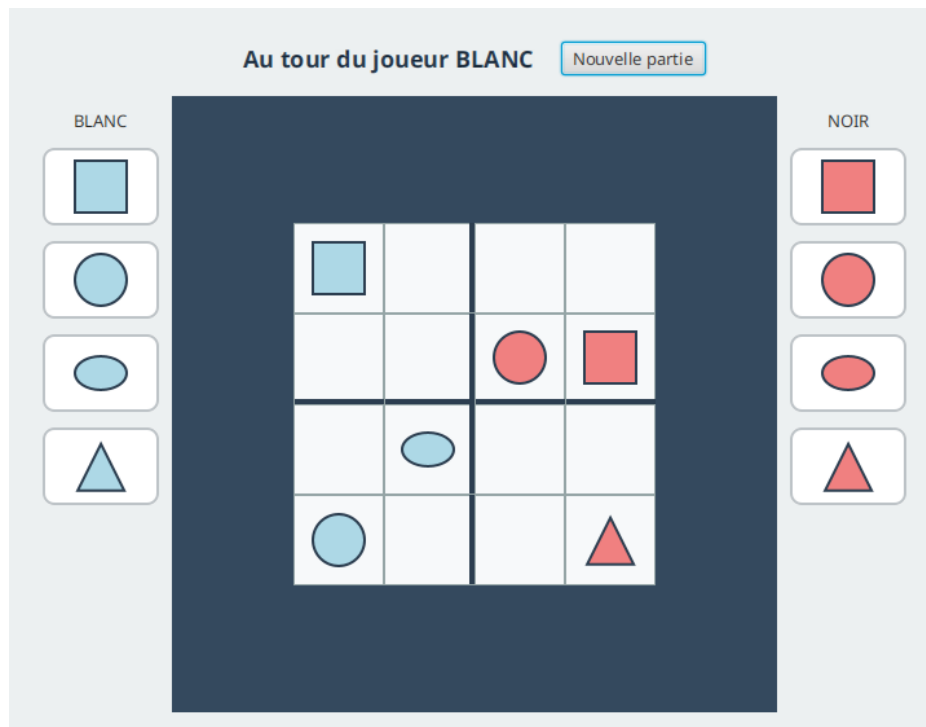
L'objectif de ce sujet est la programmation d'une version JavaFX du jeu **Quantik**.

Quantik est un jeu de stratégie abstrait édité par Gigamic (auteur : Nouredine Hilal, 2019). Deux joueurs s'affrontent sur un plateau de 4x4 cases, découpé en quatre zones de 2x2. Chaque joueur dispose de huit pièces : deux exemplaires de chacune des quatre formes (cube, sphère, cylindre, cône).

Description du jeu

On pose à tour de rôle une pièce sur une case vide en respectant une seule règle : interdit de poser une forme sur une ligne, une colonne ou une zone qui contient déjà cette forme (peu importe le joueur). Gagne celui qui pose la pièce complétant une ligne, une colonne ou une zone réunissant les quatre formes différentes.

L'IHM que vous allez réaliser ressemble à ceci : une grille 4x4 au centre (avec des bordures épaisses pour séparer les quatre zones), la réserve du joueur BLANC à gauche, celle du joueur NOIR à droite, et un bandeau de statut en haut.



Travail à réaliser

L'objectif de ce sujet est d'évaluer votre capacité à écrire une IHM en Java avec JavaFX. La logique du jeu (le "modèle") **vous est fournie** : c'est exactement le code demandé dans le sujet R2.03, livré dans le paquet `fr.univ_amu.iut.modele`. Les méthodes trop algorithmiques sont écrites pour vous. Vous pourrez retrouver une proposition de correction à l'adresse suivante : <https://github.com/IUTInfoAix-R202/TestIHM2026/>.

L'application définit plusieurs types d'objets :

- `QuantikController` est le contrôleur associé à la vue `quantikView.fxml` (fournie, avec les `fx:id`).
- `PieceRenderer` fabrique les formes JavaFX représentant les pièces.

- QuantikViewModel fait le lien entre le contrôleur et le modèle (fourni, voir ci-dessous).
- QuantikMain est l'application JavaFX qui charge la vue et affiche la fenêtre.

Vous écrirez ces classes pas à pas, en commençant par la structure du contrôleur et en terminant par l'application.

Le modèle fourni (fr.univ_amu.iut.modele) Vous pouvez vous appuyer sur les types suivants sans avoir à les écrire :

```
enum Forme { CUBE, SPHERE, CYLINDRE, CONE }
enum Joueur { BLANC, NOIR }
record Piece(Forme forme, Joueur proprietaire) {}
```

Le QuantikViewModel fourni Le ViewModel expose l'état du jeu sous forme de **propriétés observables** et de méthodes prêtes à l'emploi. Voici son interface publique :

```
public class QuantikViewModel {
    // Propriétés observables (pour les liaisons)
    ReadOnlyStringProperty statutProperty();           // ex. "Au tour du joueur BLANC"
    ReadOnlyObjectProperty<Joueur> joueurCourantProperty();
    ReadOnlyObjectProperty<Etat> etatProperty();
    ReadOnlyIntegerProperty nombreCoupsProperty();    // change à chaque coup joué
    // Lecture de l'état
    Piece pieceEn(int ligne, int colonne);            // la pièce d'une case, ou null
    int compte(Joueur joueur, Forme forme);          // pièces restantes d'un joueur
    Joueur joueurCourant();
    Forme formeSelectionnee();                        // forme choisie, ou null
    boolean estTerminee();
    String messageFin();                             // ex. "Victoire du joueur BLANC !"
    // Actions
    void selectionner(Forme forme);                   // choisir une forme à poser
    boolean peutJouerEn(int ligne, int colonne);      // le coup est-il légal ?
    boolean jouerEn(int ligne, int colonne);          // jouer (renvoie false si illégal)
    void nouvellePartie();
}
```

Exercice 1 - La structure du contrôleur

Dans cet exercice, vous commencerez par poser le squelette du contrôleur. La vue `quantikView.fxml` est fournie ci-après (ses composants portent déjà un `fx:id`).

1. Écrire la déclaration de la classe publique `QuantikController`.
2. Déclarer les cinq champs `@FXML` correspondant aux `fx:id` de la vue : le label de statut `statutLabel` (de type `Label`), le bouton `nouvellePartieBouton` (de type `Button`), les deux réserves `poolBlanc` et `poolNoir` (de type `VBox`) et le plateau `plateauGrid` (de type `GridPane`).
3. Déclarer un champ `viewModel` de type `QuantikViewModel`, initialisé avec une nouvelle instance.
4. Déclarer un tableau de `StackPane` à deux dimensions, de taille 4x4 et nommé `cases`, qui mémorisera les cases du plateau.
5. Écrire une méthode `initialize()` annotée `@FXML`, vide pour l'instant (elle sera complétée plus tard).

```

<BorderPane xmlns="http://javafx.com/javafx"
             xmlns:fx="http://javafx.com/fxml"
             fx:controller="fr.univ_amu.iut.QuantikController"
             styleClass="racine">
    <top>
        <HBox styleClass="barre-haut" alignment="CENTER" spacing="20.0">
            <children>
                <Label fx:id="statutLabel" styleClass="statut" text="Au tour du joueur BLANC" />
                <Button fx:id="nouvellePartieBouton" text="Nouvelle partie" />
            </children>
        </HBox>
    </top>
    <left>
        <VBox fx:id="poolBlanc" styleClass="pool" alignment="TOP_CENTER" spacing="12.0" />
    </left>
    <right>
        <VBox fx:id="poolNoir" styleClass="pool" alignment="TOP_CENTER" spacing="12.0" />
    </right>
    <center>
        <GridPane fx:id="plateauGrid" styleClass="plateau" alignment="CENTER" />
    </center>
</BorderPane>

```

Exercice 2 - Le rendu des pièces

La classe `PieceRenderer` transforme une `Piece` du modèle en une forme JavaFX (`Shape`) que l'on pourra afficher. Dans cet exercice, vous allez devoir utiliser les constructeurs des sous-classes de `Shape`, supposez qu'ils ont exactement les paramètres que l'on vous demande de renseigner dans l'ordre donné.

1. Écrire la méthode **statique** `Shape rendre(Piece piece)` avec un `switch` sur `piece.forme()` qui renvoie un `Rectangle` (40x40) pour `CUBE`.

```

@Test
void leCubeEstUnRectangle() {
    assertThat(PieceRenderer.rendre(new Piece(Forme.CUBE, Joueur.BLANC)))
        .assertInstanceOf(Rectangle.class);
}

```

2. Compléter le `switch` pour les autres formes : un `Circle` (rayon 20) pour `SPHERE`, une `Ellipse` (grand rayon 20 et petit rayon 13) pour `CYLINDRE`, un `Polygon` triangulaire pour `CONE`.

```

@Test
void laSphereEstUnCercle() {
    assertThat(PieceRenderer.rendre(new Piece(Forme.SPHERE, Joueur.BLANC)))
        .assertInstanceOf(Circle.class);
}

```

3. Toujours dans la méthode `rendre(Piece piece)`, après le `switch`, vous devez colorer la pièce selon son propriétaire. Pour ce faire, modifiez le remplissage avec la couleur `Color.LIGHTBLUE` pour le joueur `BLANC` et `Color.LIGHTCORAL` pour `NOIR` (méthode `setFill`).

```

@Test
void unePieceNoireEstColoreeEnRougeClair() {
    assertThat(PieceRenderer.rendre(new Piece(Forme.CUBE, Joueur.NOIR)).getFill())
        .isEqualTo(Color.LIGHTCORAL);
}

```

4. Ajouter un contour coloré à la forme en utilisant `setStroke` pour que la couleur soit grise (`Color.GREY`) et `setStrokeWidth` pour que l'épaisseur soit de deux pixels.
5. En utilisant la méthode `getStyleClass().add(...)`, ajouter à la forme la classe CSS "piece".

Exercice 3 - Le plateau

On construit la grille de jeu dans le conteneur `plateauGrid`. Les cases sont repérées par leur ligne et leur colonne, toutes deux numérotées de 0 à 3 ; la case en haut à gauche est (ligne 0, colonne 0).

1. Écrire une méthode privée `void construirePlateau()`. Elle parcourt les 16 cases avec deux boucles imbriquées. Pour chaque case : créer un `StackPane`, lui ajouter la classe CSS "case-vide" avec `getStyleClass().add(...)`, le ranger dans `cases[ligne][colonne]`, et l'ajouter à la grille avec `plateauGrid.add(cellule, colonne, ligne)`.
2. Pendant la création de chaque case, lui attacher un gestionnaire de clic avec `setOnMouseClicked` qui appelle `viewModel.jouerEn(ligne, colonne)`. Comme une expression lambda ne peut utiliser que des variables finales, recopiez d'abord `ligne` et `colonne` dans deux variables locales finales.
3. Écrire une méthode privée `void rafraichirPlateau()`. Pour chacune des 16 cases mémorisées dans `cases` : vider son contenu avec `getChildren().clear()`, récupérer la pièce avec le `viewModel` et, si elle n'est pas null, ajouter `PieceRenderer.rendre(piece)` aux enfants de la case (`getChildren().add(...)`).

Exercice 4 - L'assemblage et l'application

On réunit enfin toutes les briques dans `initialize()` (la méthode qui joue le rôle d'un constructeur pour un contrôleur FXML), puis on écrit la classe `QuantikMain`, dernière classe à réaliser.

1. Dans `initialize()`, lier le texte de `statutLabel` à la propriété de statut du `ViewModel` (méthode `bind` appelée sur `statutLabel.textProperty()`, en lui passant `viewModel.statutProperty()`), et faire en sorte qu'un clic sur `nouvellePartieBouton` appelle `viewModel.nouvellePartie()` (méthode `setOnAction`).
2. Toujours dans `initialize()`, appeler `construirePlateau()`.
3. Dans `initialize()`, écouter `viewModel.nombreCoupsProperty()` avec `addListener(...)` : à chaque changement, appeler `rafraichirPlateau()`. Appeler aussi cette méthode une fois en fin de `initialize()` pour l'affichage initial.
4. Écrire la méthode `main` la plus réduite possible pour lancer l'application (elle appelle `launch(args)`).